

Webhook Delivery Engine

Assessment - SDE

Language	Python or Node.js
Queue lib	Not permitted - implement yourself

OVERVIEW

When an event occurs in a SaaS product - a user signs up, a payment fails, a report is ready - the platform needs to notify the customer's own server via an HTTP POST request. This is called a webhook. Your task is to build the backend system that manages this delivery: accepting events, attempting HTTP delivery, retrying on failure with a defined schedule, and exposing endpoints that let customers inspect the delivery status of each event. You will not use any job queue library. Retry logic must be implemented by you.

WHAT YOU NEED TO BUILD

The system has three functional areas:

1. Event ingestion

An endpoint that accepts an incoming event from your platform and queues it for delivery to a customer's registered webhook URL. Each event has a type (e.g. payment.failed, user.signup) and a JSON payload.

2. Delivery engine

A background worker that reads pending events and attempts to POST them to the customer's endpoint. If delivery fails (non-2xx response, timeout, or connection error), the engine must retry using the following fixed schedule: retry after 30 seconds, then 5 minutes, then 30 minutes. After three failed retries the event is marked as dead. Do not use any queue library (e.g. Celery, BullMQ, RQ). Implement the scheduling and retry logic directly.

3. Status visibility

Endpoints that allow a customer to inspect the delivery status of events sent to them: list all events, retrieve a single event with its full attempt history, and manually trigger a retry for a dead event.

API ENDPOINTS

Method	Endpoint	Request body	Response
POST	/events	{ type, payload, webhook_url }	201 - event object with id and status: pending
GET	/events	-	200 - array of event objects
GET	/events/:id	-	200 - event object with attempts array
POST	/events/:id/retry	-	200 - event re-queued; 400 if not dead

Each event object must include: id, type, payload, webhook_url, status (pending / delivered / failed / dead), created_at, and an attempts array. Each attempt must include: attempted_at, http_status (or null on timeout), and outcome (success / failed).

DELIVERY RULES

- Attempt delivery as soon as the event is ingested. Do not wait for the first retry interval.
- On failure, wait the exact retry interval before the next attempt: 30 s → 5 min → 30 min.
- Any non-2xx HTTP response, a connection timeout, or a network error counts as a failure.
- After three failed retries (four total attempts), set status to dead. Stop automatic retries.
- A successful delivery (2xx response) at any attempt sets status to delivered immediately.
- Sign each outgoing request with an HMAC-SHA256 signature in the X-Webhook-Signature header. Document the signing key and verification method in your README.

CONSTRAINTS

- No queue libraries (Celery, BullMQ, RQ, Bull, Bee-Queue, or equivalent). Retry scheduling must be your own code.
- Storage: use any database or in-memory store. SQLite and in-memory are both acceptable for this assessment.
- You do not need to build a customer authentication system. Assume a single hardcoded customer for all requests.
- The delivery engine must run as a background process alongside the API server, not triggered by incoming requests.

DELIVERABLES

- Working API server and background delivery engine, runnable from your README instructions.
- README covering: setup steps, how to start the server and worker, how to verify the HMAC signature, and your retry scheduling approach.
- A note in the README on how a server restart affects in-progress retries - and whether your implementation handles it.

EVALUATION

Criterion	Weight	What we look for
Delivery engine - retry schedule is correct and self-implemented	30%	Correct intervals; no queue library used.
API correctness - all endpoints return the specified shape	25%	Status transitions are accurate; attempts logged.
HMAC signing - every outgoing request is signed correctly	20%	Signature is present and verifiable.
Failure handling - timeouts and non-2xx treated as failures	15%	Does not crash; dead status is set correctly.
README quality - setup works; restart behaviour is addressed	10%	Clear, honest, and accurate.

Instructions

Kindly create a private GitHub repository named: {yourName_Nestack_Submission} (e.g., roy_Nestack_Submission) and add the following contributors for evaluation:

- bishal@nestack.com
- sannidhya@nestack.com
- sanjay@nestack.com

Your submission must include:

- A ZIP file of your complete codebase
- A live deployment link
- Any necessary credentials if your project includes authentication

Additionally, ensure your repository includes a README.md with:

- The deployment link clearly mentioned
- Setup and run instructions if applicable

NOTE: PUBLIC REPOSITORIES ARE DISQUALIFIED FROM THE HIRING PROCESS